

HealthFlow Guinea

Audit de Production Readiness

Evaluation de la maturite du systeme pour un deploiement en production et une commercialisation

DataSphere Innovation - Sekouna KABA

14 Mai 2026

Score global : 7.3 / 10 (Prototype avance)

Niveau Production Ready : ~30% - NON PRET POUR LA PRODUCTION

Resume Executif

HealthFlow Guinea est un systeme d'information hospitalier (SIH) ambitieux et bien concu, couvrant plus de 40 modules cliniques et administratifs. Le projet presente une architecture impressionnante avec une interface utilisateur moderne (Next.js 16, React 19, Tailwind CSS, shadcn/ui), un support PWA offline-first, une internationalisation en 5 langues (Francais, Anglais, Malinke, Soussou, Poular), et une conformite HL7 FHIR R4 pour l'interopabilite. Cependant, l'audit revele que le systeme est actuellement au stade de prototype avance et presente des lacunes critiques qui empechent tout deploiement en production avec de vraies donnees patients. Les problemes majeurs incluent une base de donnees SQLite non adaptee a la production, l'absence de tests automatises, des routes API sans authentification ni validation, des integrations simulees (Orange Money, MTN MoMo, DHIS2), et des secrets codes en dur dans le code source. Ce rapport detaille chaque domaine, evalue les risques, et propose une feuille de route priorisee pour atteindre le niveau production-ready.

Matrice des Risques par Domaine

Domaine	Risque	Production Ready	Effort de correction
Securite	Critique	Non	3-5 jours
Base de donnees	Critique	Non (SQLite)	5-7 jours
Routes API	Critique	Non	10-15 jours
Authentification	Eleve	Partiel	3-5 jours
Tests	Critique	Non (0%)	15-20 jours
Integrations	Eleve	Non (simulees)	2-3 mois
Gestion d'erreurs	Eleve	Non	1-2 jours
Configuration Env	Eleve	Partiel	1 jour
Performance	Moyen	Partiel	3-5 jours
i18n	Moyen	Partiel (30%)	5-7 jours
PWA	Moyen	Non (pas d'icones)	1-2 jours
Deploiement	Moyen	Partiel	3-5 jours
Dependances	Moyen	Partiel	1 jour

Typage	Faible	Oui (95%)	0.5 jour
Logs & Monitoring	Eleve	Non	3-5 jours

1. Securite

Le module de securite de HealthFlow Guinea presente une base solide avec le chiffrement AES-256-GCM, le hachage de mots de passe bcryptjs (cost factor 12), la protection CSRF avec double-submit cookie, et une politique CSP stricte en production. Cependant, des vulnerabilites critiques subsistent qui compromettent gravement la securite du systeme en cas de deploiement reel. Ces problemes doivent etre resolus en priorite avant toute mise en production, car ils exposeraiient les donnees sensibles des patients a des risques d'acces non autorise, de fuite de donnees, et de compromission du systeme.

Points forts

Chiffrement AES-256-GCM avec Web Crypto API ; Hachage bcryptjs cost-12 avec fallback PBKDF2 ; Protection CSRF double-submit + origin validation ; CSP stricte avec nonces en production ; CORS restrictif sans wildcard ; Comparaison a temps constant anti-timing attacks

Vulnerabilites critiques

Vulnerabilite	Fichier	Detail	Impact
Cle de chiffrement en dur	security.ts:8	ENCRIPTION_KEY = process.env.ENCRIPTION_KEY 'healthflow-guinea-32byte-encryption-key'	Si la variable d'environnement est manquante, toutes les donnees sont chiffrees avec une cle publique
NEXTAUTH_SECRET fallback	auth.ts:123	Secret de dev code en dur comme fallback	Une config prod manquante utiliserait silencieusement ce secret
Rate limiting en memoire	middleware.ts	Map() pour le rate limiting	Perdu au redemarrage, ne passe pas a l'echelle horizontalement
Sessions en memoire	security.ts	Map() pour les sessions actives	Perdues au redemarrage, non partagees entre instances
secureApiHandler non applique	40+ routes	Seulement 5 routes sur 40+ utilisent le middleware securise	85%+ des routes API n'ont aucune authentication ni controle RBAC

OTP store en memoire	auth.ts	Map() pour les codes OTP	Tous les OTP actifs perdus au redemarrage
Demo user fallback	otp/route.ts	Fallback vers Dr. Mamadou Diallo si DB indisponible	Pourrait etre exploite pour obtenir un acces non autorise

2. Base de Donnees

Le schema Prisma est remarquablement complet avec plus de 50 modeles couvrant patients, rendez-vous, consultations, laboratoire, pharmacie, hospitalisation, urgences, maternite, vaccination, facturation, paiements, assurance, teleconsultation, et bien plus. Les index et contraintes sont bien definis. Cependant, le choix de SQLite comme provider de base de donnees est un bloqueur absolu pour la production. SQLite utilise un verrouillage mono-ecrivain qui empeche les ecritures concurrentes, ne supporte pas l'accès réseau, et ne dispose d'aucun mecanisme de replication. Pour un systeme hospitalier multi-utilisateurs avec des donnees de vie ou de mort, PostgreSQL est le minimum requis. Le docker-compose.yml reference deja PostgreSQL 16, mais le schema Prisma est configure pour SQLite, creant une incoherence critique qui empechera tout deploiement Docker de fonctionner.

Probleme	Risque	Detail
SQLite en production	Critique	Verrou mono-ecrivain, pas d'accès réseau, pas de replication
Incoherence Prisma/Docker	Critique	Schema SQLite mais Docker attend PostgreSQL
Pas de migrations	Eleve	Seulement prisma db push (dev-only), pas de prisma migrate deploy
Enums en String	Moyen	Pas de validation au niveau DB (status String au lieu d'enum)
Vitals en JSON String	Moyen	Donnees vitales non queryables, pas type-safe
Pas de soft-delete	Moyen	Seul Patient.isActive existe, les autres utilisent onDelete: Cascade

3. Routes API

Le projet comporte plus de 40 fichiers de routes API couvrant l'ensemble des fonctionnalites hospitalieres. Cependant, l'audit revele des lacunes systemiques majeures : seuls 8 routes sur plus de 40 disposent d'une validation Zod des entrees, et seulement 5 routes utilisent le middleware secureApiHandler qui assure l'authentification JWT, la verification CSRF, et le controle RBAC. La majorite des routes acceptent directement le body JSON sans aucune validation, ce qui expose le systeme a des injections de donnees malveillantes, des corruptions de donnees, et des acces non autorises. Dans un contexte hospitalier ou la precision des donnees patients est critique, cette absence de validation est un risque juridique et clinique majeur.

Route	Auth	Validation	RBAC	Erreurs
/api/patients	Non	Zod (POST)	Non	Try/catch
/api/billing	Non	Aucune	Non	Try/catch
/api/emergencies	Non	Aucune	Non	Try/catch
/api/payments/mobile-money	Non	Partielle	Non	Try/catch
/api/auth/otp	Non	Basique	Non	Try/catch
/api/audit	Oui	Aucune	Oui	Oui
/api/ai/preconsultation	Oui	Zod	Oui	Oui
/api/telemedicine/sessions	Oui	Partielle	Oui	Oui

4. Tests Automatises

Le constat est sans appel : le projet a un taux de couverture de tests de 0%. Il n'existe aucun fichier de test (ni .test.ts, ni .spec.ts, ni repertoire __tests__/), aucun runner de test configure (ni Jest, ni Vitest, ni Playwright), et aucun pipeline CI/CD pour l'execution automatisee des tests. Pour un systeme d'information hospitalier qui gere des donnees de sante sensibles et des processus critiques (ordonnances, resultats de laboratoire, urgences), cette absence de tests est le risque le plus eleve. Toute modification du code, meme mineure, pourrait introduire des regressions dans les workflows cliniques sans qu'aucun signal d'alerte ne soit declenche. La mise en place d'une infrastructure de test et l'ecriture de tests pour les chemins critiques (authentification, creation de patients, prescriptions, facturation) doit etre une priorite absolue.

5. Integrations Externes

Toutes les integrations externes du systeme sont actuellement simulees. Orange Money et MTN MoMo utilisent `Math.random()` pour determiner le succes ou l'echec des paiements, ce qui signifie qu'un paiement peut "reussir" ou "echouer" de maniere totalement aleatoire sans aucune transaction reelle. Le connecteur DHIS2 genere des rapports JSON mais ne les envoie jamais au systeme DHIS2 national. Les alertes mTrac, les rapports SNIS, les cartes SANTEP, et le registre national sont tous des tableaux de donnees de demonstration en dur dans le code. Cette situation est acceptable pour un prototype de demonstration mais rend le systeme inutilisable dans un contexte clinique reel. Chaque integration necessitera un travail significatif pour se connecter aux veritables API des partenaires guineens, obtenir les certifications necessaires, et mettre en place les mecanismes de gestion des erreurs et de retry pour assurer la fiabilite des echanges.

Integration	Statut	Detail
Orange Money	Simulee	<code>Math.random()</code> pour succes/echec, <code>delay()</code> pour la latence
MTN MoMo	Simulee	Meme simulation qu'Orange Money
DHIS2	Types uniquement	<code>generateDHIS2Report()</code> genere du JSON sans l'envoyer
SNIS	Donnees demo	<code>demoSNISReports</code> - tableau en dur
mTrac	Donnees demo	<code>demoMTracAlerts</code> - tableau en dur
SANTEP Card	Donnees demo	<code>demoSANTEPCards</code> - tableau en dur
Registre National	Donnees demo	<code>demoNationalRegistry</code> - tableau en dur
SMS (Orange)	Code existe	Requiert <code>ORANGE_SMS_TOKEN</code> (non configure)
SMS (Twilio)	Code existe	Requiert identifiants Twilio (non configures)

6. Autres Domaines

6.1 Authentification

L'authentification repose sur NextAuth v4 avec un fournisseur Credentials (telephone + OTP). Le systeme OTP est bien conçu (6 chiffres, expiration 5 min, max 3 tentatives, rate limiting), mais le stockage des OTP en memoire (Map()) et le fallback vers un utilisateur demo lorsque la base de donnees est indisponible sont des risques critiques. Il n'existe pas de mecanisme de verrouillage de compte, pas d'authentification MFA active (bien que le schema la supporte), pas d'authentification par mot de passe (uniquement OTP, ce qui pose probleme si la passerelle SMS est indisponible), et aucune possibilite de revoke une session JWT active.

6.2 Gestion d'Erreurs

Le projet ne dispose d'aucune page `error.tsx`, `not-found.tsx`, ou `global-error.tsx`. Les erreurs React non gérées provoquent le plantage complet de l'application sans aucun message utilisateur. Cote API, les erreurs peuvent fuir des details internes (`err.message` expose directement), ce qui constitue un risque de securite. L'absence de service de suivi des erreurs (Sentry, Datadog) signifie que les erreurs en production ne seraient detectees que par les plaintes des utilisateurs, ce qui est inacceptable pour un systeme hospitalier.

6.3 Configuration Environnement

Le fichier `.env` ne contient que `DATABASE_URL`. Il n'existe pas de fichier `.env.example` pour guider les nouveaux developpeurs, pas de `.env.production` ou `.env.staging`, et aucune validation des variables d'environnement au demarrage. Les secrets critiques (`ENCRYPTION_KEY`, `NEXTAUTH_SECRET`) ont des fallbacks codes en dur, ce qui signifie que l'application démarre silencieusement avec des valeurs insecure si les variables ne sont pas definies. Cette situation est particulierement dangereuse car aucune alerte n'est emise pour signaler la configuration insecure.

6.4 Performance

Les routes API supportent la pagination via `getPaginationParams()` et `PrismaClient` est correctement singletonise. Cependant, Redis est configure dans Docker Compose mais jamais utilise dans le code pour le cache. Chaque requete API atteint directement la base de donnees sans aucune mise en cache. Le logging

Prisma est actif dans tous les environnements (log: ['query']), ce qui ajoute un overhead significatif en production. Le store Zustand data-store.ts charge toutes les données de démonstration en mémoire d'un coup, ce qui ne passe pas à l'échelle avec des données réelles.

6.5 Internationalisation (i18n)

Le système supporte 5 langues, mais les langues autochtones (Malinke, Soussou, Poular) ne sont complètes qu'à environ 30% (136-184 lignes contre 499 pour le Français et l'Anglais). Les sections analytics, administration, settings, et patient-portal sont particulièrement incomplètes dans les langues locales. La bibliothèque next-intl est listée comme dépendance mais n'est pas utilisée ; un système i18n personnalisé est implémenté à la place. Il n'existe pas de validation des clés de traduction manquantes, ce qui signifie que les clés absentes s'affichent telles quelles ou sont remplacées silencieusement.

6.6 Déploiement

Le Dockerfile est bien structuré (multi-stage, utilisateur non-root, health check, standalone output). Le docker-compose.yml configure PostgreSQL 16, Redis 7, et Nginx avec des health checks. Cependant, l'incohérence critique entre le schéma Prisma (SQLite) et l'infrastructure Docker (PostgreSQL) signifie que le déploiement échouera immédiatement. Il n'y a pas d'étape de migration de base de données dans le Dockerfile ou le script de démarrage. Le Caddyfile n'a pas de configuration HTTPS/TLS, et il n'existe pas de procédure de backup/restore des données, ce qui est inacceptable pour un système de santé.

6.7 Logs et Monitoring

Le endpoint /api/health retourne l'uptime, l'utilisation mémoire, et l'environnement. Le module audit-logger.ts offre des types d'événements comprehensifs et des fonctions de journalisation. Cependant, les journaux d'audit sont stockés uniquement en mémoire (max 10 000 entrées, perdus au redémarrage), le code indique explicitement "remplacer par une base de données en production". Il n'y a pas de logging structuré (uniquement console.log/warn/error), pas de service de monitoring externe (Sentry, Datadog), pas de vérification de la connectivité base de données dans le health check, pas d'endpoint de métriques Prometheus, et pas de système d'alerte pour les événements critiques.

7. Feuille de Route vers la Production

La feuille de route suivante presente les phases priorises pour transformer HealthFlow Guinea d'un prototype avance en un systeme production-ready. L'estimation totale est de 2 a 3 mois avec une equipe dediee de 2 a 3 developpeurs. Les phases sont organisees par ordre de priorite, chaque phase construisant sur la precedente. Il est imperatif de ne pas sauter de phase : la securite et la base de donnees doivent etre resolues avant de travailler sur les integrations ou les tests avances.

Phase 1 : Durcissement Securitaire (1 semaine)

Action	Priorite	Effort
Supprimer tous les fallbacks de secrets en dur	Critique	0.5 jour
Ajouter le rate limiting Redis	Critique	1 jour
Appliquer secureApiHandler a toutes les routes API	Critique	2 jours
Migrer le store OTP vers Redis	Eleve	0.5 jour
Ajouter la validation Zod a toutes les routes POST/PUT	Critique	1.5 jours
Supprimer le fallback utilisateur demo	Critique	0.5 jour

Phase 2 : Migration Base de Donnees (1 semaine)

Action	Priorite	Effort
Basculer Prisma de SQLite vers PostgreSQL	Critique	1 jour
Creer les migrations prisma migrate dev	Critique	1 jour
Ajouter la verification DB au health check	Eleve	0.5 jour
Convertir les enums String en enums Prisma	Moyen	1 jour
Configurer les sauvegardes automatiques	Eleve	1 jour
Mettre en place le connection pooling	Moyen	0.5 jour

Phase 3 : Fiabilite (2 semaines)

Action	Priorite	Effort
--------	----------	--------

Ajouter error.tsx, not-found.tsx, global-error.tsx	Eleve	0.5 jour
Implementer le logging structure (pino/winston)	Eleve	1 jour
Migrer les audits logs en base de donnees	Eleve	1 jour
Integrer Sentry pour le suivi des erreurs	Eleve	0.5 jour
Configurer Vitest + tests unitaires chemins critiques	Critique	5 jours
Generer les icones PWA et la page offline	Moyen	0.5 jour
Ajouter les tests E2E Playwright (5 scenarios)	Eleve	3 jours

Phase 4 : Integrations Reelles (2-3 mois)

Action	Priorite	Effort
Integrer l'API reelle Orange Money	Critique	2 semaines
Integrer l'API reelle MTN MoMo	Critique	2 semaines
Integrer la passerelle SMS reelle	Eleve	1 semaine
Connecter au DHIS2 national pour les rapports	Eleve	2 semaines
Ajouter la verification de signature webhook	Critique	1 semaine
Mettre en place les mecanismes de retry	Eleve	1 semaine

8. Conclusion et Recommandations

HealthFlow Guinea presente un potentiel considerable comme systeme d'information hospitalier pour la Guinee et l'Afrique de l'Ouest. L'architecture est moderne, les modules sont comprehensifs, l'interface utilisateur est soignee, et la conformite HL7 FHIR R4 est un atout majeur pour l'interopabilite regionale. Cependant, le systeme est actuellement au stade de prototype avance avec un niveau de maturite production d'environ 30%. Les 4 bloqueurs absolus pour la production sont : (1) la base de donnees SQLite qui ne supporte pas la concurrence, (2) l'absence de tests automatises qui rend chaque modification risquee, (3) les routes API sans authentication qui exposent les donnees patients, et (4) les integrations simulees qui empechent les transactions reelles.

La recommandation principale est de suivre la feuille de route en 4 phases, en commençant impérativement par le durcissement securitaire et la migration PostgreSQL. Avec un investissement de 2 a 3 mois et une equipe dediee, HealthFlow Guinea peut atteindre le niveau production-ready et devenir un veritable leader de la sante numerique en Afrique francophone. L'approche progressive est essentielle : chaque phase doit etre validee avant de passer a la suivante, et les tests automatises doivent accompagner chaque developpement des la Phase 3 pour garantir la non-regression du systeme.

Carte de Score Final

Domaine	Score	Commentaire
Fonctionnalite (couverture modules)	9/10	40+ modules couvrant tout le spectre hospitalier
Architecture technique	8/10	Next.js 16, FHIR R4, PWA, i18n - solide et moderne
Securite	4/10	Bonne base mais secrets en dur, routes non protegees
Base de donnees	3/10	Schema excellent mais SQLite bloque la production
Tests	0/10	Aucun test - risque de regression maximal
Integrations	1/10	Toutes simulees - aucune transaction reelle possible
Deploiement	5/10	Docker OK mais incoherence SQLite/PostgreSQL

Production Readiness	3/10	Prototype avance - travail significatif requis
----------------------	------	--